BLUE HARVEST

Mortgage Pricing Exception Requests

Agentic SDLC Engineering Assignment

Presented by Ferdinand Dinga

8 November 2026

What this presentation covers

From the client problem to a runnable solution and an engineering handoff.

01 Understand the assignment

Client needs, the approval problem, and the proposed solution.

02 Explain the implementation

Main workflow, architecture, technology choices, and design decisions.

03 Run and validate it

Docker setup, test commands, delivered artefacts, and live demonstration.

Goal for the walkthrough

Show how the solution meets the brief and how another developer can run, verify, and continue it.

A discount must be reviewed before it can be used

Relationship managers need flexibility; the mortgage process needs an unambiguous, auditable approval.

Required business behaviour

Create a request using an application ID, discount in basis points, and reason.

Record approval or decline, reviewer identity, and decision time.

Required operational behaviour

Expose the current approved discount and request history.

Handle retries after timeouts without duplicates; run locally without private infrastructure.

The key tension in the brief

Managers may change submitted terms, but an approval must always belong to the exact terms reviewed.

A small API, a usable UI, and durable workflow records

An internal React application backed by ASP.NET Core and PostgreSQL.

Relationship manager

Creates and revises requests.
Selects an independent reviewer.
Sees current status and history.

Assigned reviewer

Reviews the current terms.
Approves or declines with a note.
Decision records actor and time.

Mortgage process

Retrieves the effective discount.
Only a current approval applies.
Revised terms require reapproval.

Local demonstration identities

3 managers + 5 reviewers + Mortgage Process; existing development header authentication, with no registration.

Why store an application ID instead of a full mortgage?

This service owns pricing exceptions; the mortgage application is assumed to belong to an external system.

Why this scope was chosen

The brief asks for an application identifier and allows us to choose its representation.

No upstream mortgage API or application data contract is supplied.

What the ID gives us

Links a request to its mortgage application without duplicating customer or loan records.

Keeps the exception workflow small and independently runnable.

What is not verified yet

The API checks for a non-empty GUID; it does not prove the application exists or is eligible.

Production must validate it through the owning mortgage system.

Explicit assumption—not a completed integration

Demo IDs are references only. Do not interpret a successful exception request as proof of a valid mortgage application.

Separate assignment requirements from chosen policies

These exercise choices need stakeholder confirmation before production use.

One request per application	Use revisions for later proposals, including after decline.	Keeps one decision history; confirm whether parallel requests are needed.
Discount: integer 1–500 bps	Use a simple bounded range with no fractional basis points.	A demo validation policy, not a banking limit specified by the brief.
Independent assigned reviewer	Only the selected reviewer decides; creator cannot self-review.	Provides clear accountability; confirm delegation and approval limits.
Reasons and read visibility	Request reason required; review note optional; known users can read.	Supports the demo; confirm mandatory decline reasons and privacy rules.
Approval lifetime	Approval stays effective until a new revision; no expiry or consumption.	No lifecycle contract was supplied; agree expiry and downstream use.

Create → assign → review → retrieve

A clear decision path, with revisions that preserve the earlier terms and decisions.

1

Create

Application ID
Discount + reason

2

Assign

Select one of
five reviewers

3

Decide

Approve or decline
Actor + UTC time

4

Retrieve

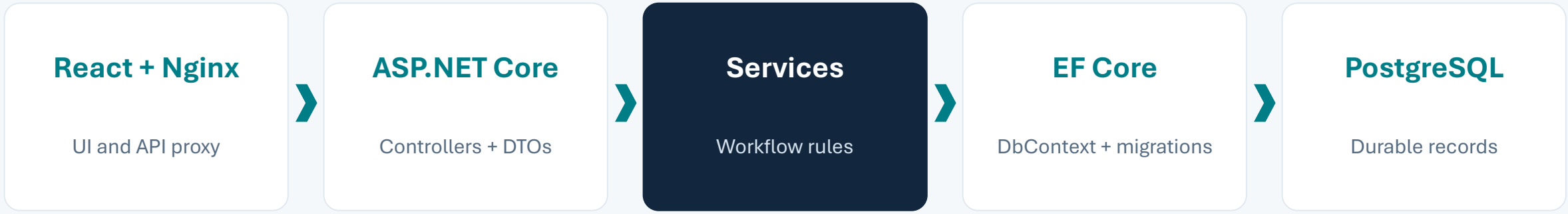
Effective discount
for current terms

If the manager changes the terms

Create a new revision → return to Pending → withdraw the effective approval → obtain a fresh decision.

Separate HTTP handling, workflow rules, and persistence

One backend application with clear logical boundaries, deployed alongside the frontend and database.



Service responsibilities

PricingExceptionService handles requests and decisions.
UserService checks identities and roles.
ApprovedDiscountService exposes effective discounts.

Compose deployment

Frontend + backend + PostgreSQL.
A named volume persists database data.
EF migrations run at API startup.

Why these technologies fit this assignment

Rationale and trade-offs for a small, testable local solution.

React + TypeScript	Reusable forms, typed API contracts, and responsive user interactions.	Adds a frontend build; the API remains usable through Swagger.
ASP.NET Core / C#	Built-in DI, validation, HTTP APIs, and a language familiar to the team.	Keep the application small rather than adding unnecessary services.
EF Core + PostgreSQL	Explicit migrations, durable data, and transactional saves.	Needs a database container; relational checks need live validation.
Docker Compose	Repeatable local startup for the UI, API, and database.	Requires Docker Desktop and available host ports.

Resolve changed terms without rewriting an approval

Explicit rules protect review integrity, request history, and retry behaviour.

Immutable revisions

New terms create a new revision. The old decision stays in history but stops authorising the new terms.

Idempotent writes

A retry with the same key and input replays the original result; conflicting reuse returns 409.

Version checks + audit trail

Stale edits are rejected. Actor IDs, name snapshots, reasons, and UTC times retain attribution.

Dependency injection

Scoped services and DbContext separate responsibilities. Interfaces and TimeProvider support testing.

From repository files to a running application

Run these commands from the repository root—the folder containing compose.yaml.

1 Install prerequisites

Install and start Docker Desktop using Linux containers.

Git is optional if downloading a ZIP. No host Node.js is needed for Compose.

2 Get the source

Clone the shared Git repository, or download and extract its ZIP.

Open a terminal in the extracted root.
Keep the full folder structure.

3 Build and start

Compose builds the frontend and API, starts PostgreSQL, applies migrations, and seeds demo users.

```
docker compose up --build -d
docker compose ps
```

Frontend http://localhost:5173
Swagger http://localhost:5267/swagger

Verify the workflow and preserve the database

Install the .NET 10 SDK to run host-side backend tests; keep Docker running for the live demo.

Backend tests (from repository root)

```
cd Backend/BlueHarvest.PricingExceptions
dotnet test
```

Verify in the browser

Open the frontend and select James.
Create a request and assign a reviewer.
Switch users, decide, and inspect history.

From the repository root

```
docker compose logs --tail 50 backend postgres frontend
docker compose restart
docker compose stop
```

Use docker compose up -d to start again. The named volume keeps data; down -v deletes it.

Test evidence supports the implementation choices

Recorded local results: isolated automated tests plus real Docker/PostgreSQL workflow checks.

184 .NET tests passed

56 unit + 118 HTTP + 10 PostgreSQL cases.

All manager/reviewer combinations, permissions, revisions, history, retries, and discount retrieval.

20 browser tests passed

UI forms, user switching, decision attribution, and responsive layout.

The three-request flow also passed against Compose/PostgreSQL.

Live deployment checks

The API smoke test exercises the real PostgreSQL-backed workflow.

Persistence and reseeding checks verify retained records and stable users.

Additional live checks

API smoke tests, persistence across recreation, and restart reseeding without duplicate users or record loss.

How Codex contributed—and how output was checked

AI supported the engineering workflow; the developer directed scope and remains accountable for the result.

Implementation assistance

Drafted and refined API and UI code, DTOs, service rules, seed data, and explanatory comments.

Helped diagnose proxy ports, package conflicts, and Docker DNS issues.

Validation + handoff

Generated and executed unit, HTTP, and browser tests; used failures to guide corrections.

Prepared Docker configuration, runbooks, diagrams, and presentation material.

Developer controls

Work used developer-directed steps and checkpoints.

Review generated changes, check requirements independently, and rerun tests.

Limits and remaining risk

AI can miss requirements or generate matching code/test defects. Humans make mortgage decisions; no AI approval agent is implemented.

Priorities for the developers who take this forward

Complete these controls before moving beyond the local demonstration.

1 Identity + external integration

Integrate enterprise SSO; validate JWTs and roles.
Remove demo switching from production.
Validate IDs and eligibility with the mortgage service.
Confirm discount and approval policies.

2 Database + concurrency

Add database uniqueness and retry constraints.
Design transactions for multi-instance races.
Test against PostgreSQL in CI.
Plan and verify safe schema upgrades.

3 Deployment + recovery

Move credentials to managed secrets.
Configure HTTPS and secure ingress.
Automate builds, checks, and deployment.
Test backup restoration and rollback.

4 Operations + governance

Export OpenTelemetry traces to AWS X-Ray.
Use CloudWatch or Prometheus/Grafana for monitoring.
Run load, security, and recovery tests.
Assign support ownership and runbooks.

Assumptions and production improvements

Current assumption	Proposed production improvement
Mortgage applications are external; only IDs are stored.	Integrate the mortgage service and validate application existence and eligibility.
Users are selected from a demo dropdown.	Use enterprise SSO such as Cognito, validate JWTs and enforce roles.
OpenTelemetry is present without production observability infrastructure.	Export traces to AWS X-Ray; use CloudWatch or Prometheus and Grafana for metrics, dashboards and alerts.

Future improvements, not completed integrations.

A clear handoff for the next engineer

The delivered artefacts address the brief; remaining production work is explicitly scoped.

Delivered in the repository

Source code + Docker setup + run instructions.
README, assumptions, diagrams, and EF migrations.
Automated tests and operational test scripts.
React UI bonus and AI contribution record.

Before production

Trusted identity and authorisation integration.
Real mortgage-application existence checks.
Multi-instance concurrency guarantees.
Backups, observability, and operational ownership.

Before submission

Confirm the reviewer can access the Git repository and include the actual time spent, as requested in the brief.

Request actions and their service calls

Current names for creation, review and revision; the routes remain unchanged.

Create a request — POST /pricing-exceptions

Controller: CreatePricingExceptionRequestAsync
Service: CreatePricingExceptionRequestAsync

Record a decision — POST /{id}/decisions

Controller: RecordPricingExceptionDecisionAsync
Service: ReviewPricingExceptionRequestAsync

Submit a revision — POST /{id}/revisions

Controller: SubmitPricingExceptionRevisionAsync
Service: SubmitRequestRevisionAsync

Reads, identities and approved discounts

Action → service method. Every call retains its existing cancellation token and access checks.

CONTROLLER ACTION	SERVICE METHOD
GetPricingExceptionRequestsAsync	ListPricingExceptionRequestsAsync
GetPricingExceptionRequestByIdAsync	GetPricingExceptionRequestAsync
GetPricingExceptionHistoryAsync	GetRequestHistorySnapshotAsync
GetCurrentUserProfileAsync	GetUserOrThrowAsync
GetReviewersAsync	GetReviewersAsync
GetApprovedDiscountForMortgageApplicationAsync	ApprovedDiscountService.GetAsync

Helpers make the workflow responsibilities explicit

Private helpers remain below public methods. UserService preserves stored identity and role validation.

PricingExceptionService

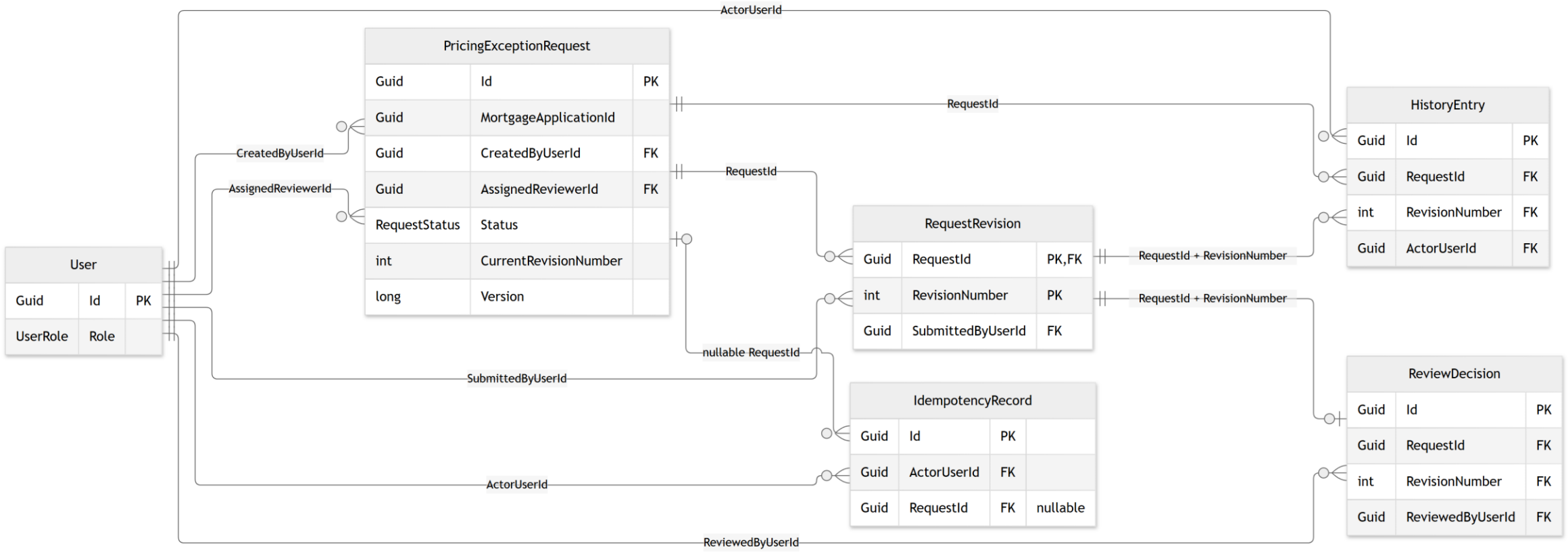
ExecuteAuthorizedRequestReadAsync
FindRequestOrThrowAsync
ExecuteIdempotentWriteAsync
ValidateDiscountTerms
ValidateRequestVersionAndRevision
CreateRequestRevision
AppendRequestHistoryEntry
MapToPricingExceptionResult

UserService

GetUserOrThrowAsync
GetUserWithRequiredRoleAsync
GetReviewersAsync

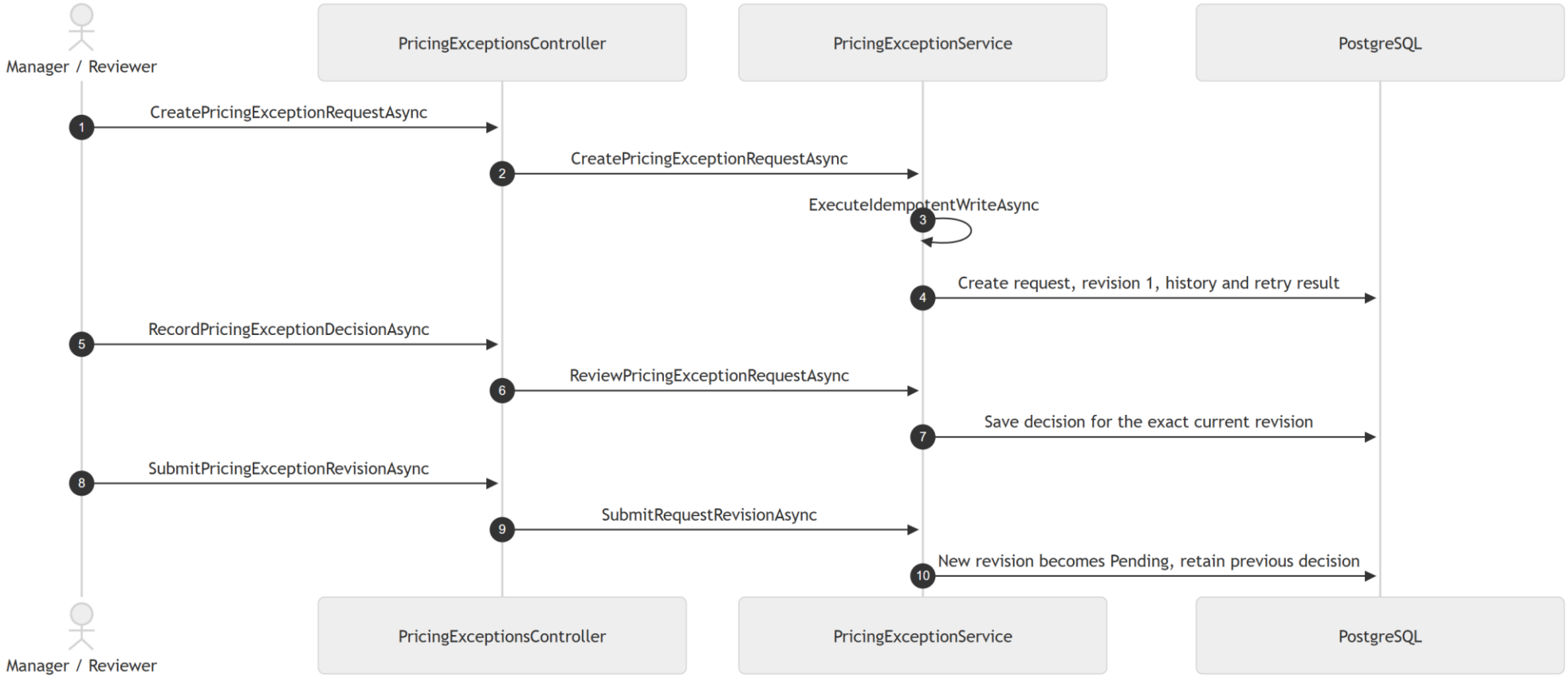
Entity relationships remain unchanged

Six persisted entities. The naming refactor changes methods, not tables, properties or constraints.



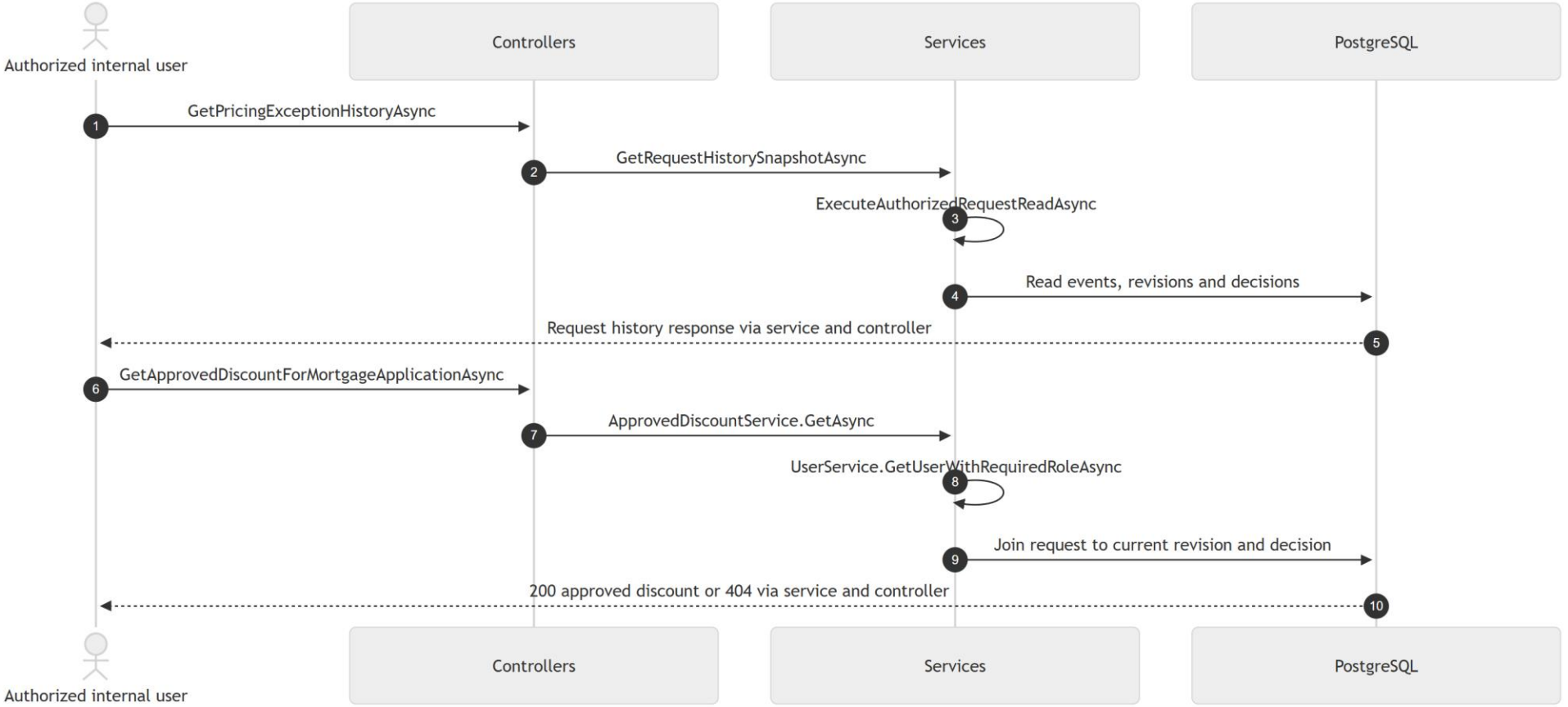
Sequence: submit, decide and revise

Current controller actions and service messages; each new revision requires a fresh decision.



Sequence: history and approved-discount lookup

Current read methods retain identity checks, history snapshots and effective-approval rules.



Thank you

Any questions?

Ferdinand Dinga | 8 November 2026

Live demo: create → assign → decide → inspect history